Enabling Quantum Mechanics for Materials Science-2: Implementation of Direct Inversion of the Iterative Subspace (DIIS) and its Performance

Fredy Aquino

October 21st, 2014

Contents

1 Introduction **1**

2 Computational and Implementation Details **2**
2.1 **D**irect **I**nversion of the **I**terative **S**ubspace (**DIIS**) 2
2.2 Implementation of DIIS in linear scaling CP2K 3

3 Results and Discussion **5**

4 Summary and conclusions **11**

A Modified ls_scf_main routine **13**

B qs_diis_b_step_4lscf routine **19**

C LS+DIIS input test for closed shell CH_4 **24**

D LS+DIIS input test for open shell CHO **25**

Abstract

The Implementation of **D**irect **I**nversion of the **I**terative **S**ubspace (**DIIS**) within linear scaling module in CP2K software shows expected energy convergence acceleration without affecting linear scalability (LS) with the additional benefit of ensuring convergence to ground state.

Keywords: DIIS, Linear Scaling, DFT, DFTB

1 Introduction

Linear scaling methods in Quantum Chemistry are aimed into computations on large molecular systems (e.g. 100 K atoms to millions of atoms) for which the usage of

standard diagonalization method is inappropriate since it scales as N^3 where N is the number of atoms. A linear scaling method takes advantage of the *Nearsightedness Principle*[8,10] that says that the interactions are limited to a local volume allowing us to represent a Kohn-Sham matrix, K, and density matrix, P, using sparse matrices where most of the off-diagonal elements are nearly negligible. Also, in linear scaling framework we avoid using diagonalization method that computes the molecular orbital coefficients, $\mathbf{C}$, and therefore the density matrix, $\mathbf{P} = \mathbf{C}\mathbf{C}^{\dagger}$. Instead, we use a density matrix purification method (e.g TRS4[9]) based on a polynomial transformation, $\mathbf{P}_{i+1} = \mathcal{F}(\mathbf{P}_i)$, to reach idempotency within certain threshold. Among several linear scaling implementations we adopted the one in CP2K software[13,14] for several reasons: i) it is a free open source code; ii) it can simulate several thousands of atoms within picoseconds with HPC resources; iii) it provides several flavors of Molecular Dynamics (MD) ensembles (e.g. NVE, NVT, NPT, MSST); iv) uses advantageous localized Gaussian orbitals augmented with plane waves and other basis sets; v) it shows a promising order N framework with growing features and available support.

The approximations adopted in linear scaling method (e.g. CP2K) imply certain margin of error with respect to diagonalization method depending on the settings we adopt.

In the linear scaling CP2K there is a rudimentary way to speed up energy convergence from mixing two previous Kohn-Sham matrices using a MIXING_FRACTION parameter: K'= A * K + (1-A)K_{old} (with MIXING_FRACTION=A). We decide to enhance that way of speed up energy convergence by replacing it, upon choice of the user, by a more sophisticated way to converge doing an extrapolation of previous Kohn-Sham matrices which are combined linearly using weighting coefficients that are computed using **D**irect **I**nversion of the **I**terative **S**ubspace (**DIIS**).[5,11,12] This report is showing details on implementation of DIIS in linear scaling CP2K and its performance.

2 Computational and Implementation Details

In this section we will give an outline of the **D**irect **I**nversion of the **I**terative **S**ubspace (**DIIS**) method and how it is adapted into the linear scaling module in CP2K.

2.1 Direct Inversion of the Iterative Subspace (DIIS)

The **D**irect **I**nversion of the **I**terative **S**ubspace (**DIIS**) minimizes the square of the norm of the best estimate of the Kohn-Sham matrix error $f^{(j)}$, $\left(\sum_{j=1,N_{\text{buffer}}} f^{(j)} c_j\right)^2$, under the constraint that the summation of DIIS weighting coefficients, c_j, equal unity[5,11,12]

$$W = \left(\sum_{j=1,N_{\text{buffer}}} f^{(j)} c_j\right)^2 + 2\lambda\left(\sum_{j=1,N_{\text{buffer}}} c_j - 1\right) \tag{1}$$

$$\frac{\partial W}{\partial c_i} = 2\sum_{j=1,N_{\text{buffer}}} \left\langle f^{(i)} \middle| f^{(j)} \right\rangle c_j + 2\lambda = 0. \tag{2}$$

From Equation (2) one obtains the DIIS weighting coefficients from solving the following linear system,

$$\begin{bmatrix} \mathbf{B} & \mathbf{1} \\ \mathbf{1} & \mathbf{0} \end{bmatrix} \begin{bmatrix} c \\ \lambda \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ 1 \end{bmatrix} \tag{3}$$

where $\mathbf{B}$ has a dimension $N_{\text{buffer}} \times N_{\text{buffer}}$ and its matrix elements, b_{ij}, are

$$b_{ij} = \left\langle f^{(i)} \middle| f^{(j)} \right\rangle \tag{4}$$

with $f^{(i)}$ defined as

$$f^{(i)} = \begin{cases} \mathbf{K}^{(i)}\mathbf{P}^{(i)}\mathbf{S} - \mathbf{S}\mathbf{P}^{(i)}\mathbf{K}^{(i)}, & \text{if non-orthogonal orbitals} \\ \mathbf{K}^{(i)}\mathbf{P}^{(i)} - \mathbf{P}^{(i)}\mathbf{K}^{(i)}, & \text{if orthogonal orbitals.} \end{cases} \tag{5}$$

For a case of $N_{\text{buffer}} = 4$ Equation (3) can be explicitly shown as

$$\begin{bmatrix} \left\langle f^{(1)} \middle| f^{(1)} \right\rangle & \left\langle f^{(1)} \middle| f^{(2)} \right\rangle & \left\langle f^{(1)} \middle| f^{(3)} \right\rangle & \left\langle f^{(1)} \middle| f^{(4)} \right\rangle & 1 \\ \left\langle f^{(2)} \middle| f^{(1)} \right\rangle & \left\langle f^{(2)} \middle| f^{(2)} \right\rangle & \left\langle f^{(2)} \middle| f^{(3)} \right\rangle & \left\langle f^{(2)} \middle| f^{(4)} \right\rangle & 1 \\ \left\langle f^{(3)} \middle| f^{(1)} \right\rangle & \left\langle f^{(3)} \middle| f^{(2)} \right\rangle & \left\langle f^{(3)} \middle| f^{(3)} \right\rangle & \left\langle f^{(3)} \middle| f^{(4)} \right\rangle & 1 \\ \left\langle f^{(4)} \middle| f^{(1)} \right\rangle & \left\langle f^{(4)} \middle| f^{(2)} \right\rangle & \left\langle f^{(4)} \middle| f^{(3)} \right\rangle & \left\langle f^{(4)} \middle| f^{(4)} \right\rangle & 1 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}. \tag{6}$$

Using those DIIS weighting coefficients one can obtain an *improved* Kohn-Sham matrix, K, using previous Kohn-Sham matrices from their extrapolation in the following way,

$$\mathbf{K}' = \sum_{j=1,N_{\text{buffer}}} c_j \mathbf{K}^{(j)} \tag{7}$$

2.2 Implementation of DIIS in linear scaling CP2K

The DIIS method is already implemented in CP2K for diagonalization based SCF where it makes use of full matrices (fm) and the density matrix is constructed within DIIS routine from the usage of molecular orbital (MO) coefficients $\mathbf{C}$, $\mathbf{P} = \mathbf{C}\mathbf{C}^{\dagger}$.

Linear scaling module in CP2K is aiming to work on large molecular systems (e.g 100 K atoms up to millions of atoms) and for that purpose its implementation and performance relies on the usage of a sparse matrix library called DBCSR library.[1]

The implementation of DIIS within linear scaling module in CP2K adapts the already implemented code of DIIS making the following important changes: i) Input density matrix $\mathbf{P}$ instead of MO coefficients $\mathbf{C}$; ii) redefine matrix variables using available structures in linear scaling environment (e.g. cp_dbcsr_type used mainly to hold sparse matrices); iii) re-definition of DIIS routines within linear scaling environment (e.g qs_diis_b_clear_sparse is the sparse version of qs_diis_b_clear, qs_diis_b_create_sparse is the sparse version of qs_diis_b_create, qs_diis_b_step_4lscf is the sparse version of qs_diis_b_step) making use of DBCSR library.

Below is shown the pseudocode of the main routine in linear scaling CP2K, ls_scf_main, after adding DIIS:

```
 PSEUDOCODE for ls_scf_main() [In dm_ls_scf.F]:
IF (ls_diis) THEN
1. Create+Clear DIIS buffer, Copy overlap matrix
ENDIF
DO ! the real SCF loop ----- START
 (get time,iscf=iscf+1)
 Get a copy of the current KS matrix
 IF(iscf>1 .AND. curvy_steps) THEN
  Run curvy steps
 ELSE
  Turn the K matrix in a density matrix
  DO ispin=1,nspin
   IF (iscf==1) THEN
    Copy initial Kohn-Sham matrix, K
   ELSE ! mix-DIIS or mix-Fraction --- START
    2. Use mix-DIIS or mix-Fraction for K matrix:
    IF (ls_diis) THEN
     IF (diis_step.and.(iscf-1).ge. iter_ini_diis) THEN
      Use DIIS mixed K    :K = Linear Combination K_old's
     ELSE
      Use MIXING_FRACTION=A:K = A*K+(1-A)*K_old
     ENDIF
    ELSE ! otherwise
      Use MIXING_FRACTION=A:K = A*K+(1-A)*K_old
    ENDIF
   ENDIF ! mix-DIIS or mix-Fraction --- END
   Purify density matrix
   Preconditioning density matrix and filtering
  ENDDO ! end-loop-ispin
 END IF
 Compute new (K matrix,energy)
 Report status of energy convergence
 Exit criterium on the energy
 IF (ls_diis) THEN
   3. Call DIIS to update c_i coefficients:
  CALL qs_diis_b_step_4lscf
 ENDIF
ENDDO  ! the real SCF loop ----- END
 4. Free DIIS buffer storage
```

and below is shown the pseudocode for the main DIIS routine called qs_diis_b_step_4lscf which does the following: i) construct *on the fly* the DIIS buffer **B** (Equation (3)); ii) finds the DIIS weighting coefficients, c_i, from solving the linear system shown in Equation (3); iii) updates the Kohn-Sham matrix using a history of previous Kohn-Sham matrices and those found DIIS weighting coefficients (Equation (7)).

```
PSEUDOCODE for qs_diis_b_step_4lscf():
1. Getting current Kohn-Sham matrix
2. Allocate DIIS buffer + Create scratch arrays
3. Compute Kohn-Sham error error_max:
  DO ispin=1,nspin
   IF (PRESENT(S)) THEN
     E = K*P*S - transpose(K*P*S)
   ELSE
     E = K*P   - transpose(K*P)
   END IF
   Compute error_max=MAX(E)
  END DO
IF (error_max < eps_diis) THEN
 4. Update the LS SCF DIIS buffer, B
ENDIF
5. Perform DIIS step or update K_old(ispin,i):
IF (diis_step) THEN ! Perform DIIS step
 Set up + Solve the DIIS linear system (Eq. 3)
 IF (iscf.GE.iter_ini_diis) THEN ! Update K matrix
   K(ispin=1,nspin)= �sum_{i=1,nb} c_i K_old(ispin,i)
 ENDIF
ELSE
  Add K(ispin=1,nspin) to K_old(ispin,i)
END IF
```

3 Results and Discussion

Upon crafting the new linear scaling source code in CP2K we would like to do quick tests in order to check if we gain energy convergence acceleration within linear scaling module in CP2K. The first two tests we can think of are to do a DFT calculation on very small system and to count the number SCF cycles with and without DIIS. It is important to point out that these tests are unrealistic in the sense that nobody should attempt to use linear scaling feature when dealing with small molecular systems, we could easily do it using standard diagonalization method. We did the following two tests:

1. Closed shell system: Methane, CH_4. Single energy point calculation using DFT with exchange- correlation functional PADE[4] and DZVP-GTH-PADE basis set.

2. Open shell system: CHO. Single energy point calculation using DFT with exchange-correlation functional PADE and DZVP-GTH-PADE basis set.

The results of these tests are shown in Table (1) for CH_4 and Table (2) for CHO, respectively, where we can see that the effect of adding DIIS in linear scaling DFT in CP2K is to speed up energy convergence by reducing the number of SCF cycles.

Table 1: Comparison of single point energy convergence calculations on closed shell CH_4 using linear scaling DFT[a] with (XC,BS)=(PADE,DZVP-GTH-PADE) with and without DIIS.

Without DIIS[b]		
SCF Nr	Energy (au)	ΔE (au)
1	-7.880518629	-7.880518629
2	-7.993215364	-0.112696734
3	-7.994758431	-0.001543068
4	-7.994969323	-0.000210892
5	-7.995014223	-0.000044900
6	-7.995024006	-0.000009783
7	-7.995026143	-0.000002138
With DIIS[c]		
SCF Nr	Energy (au)	ΔE (au)
1	-7.880518629	-7.880518629
2	-7.993215364	-0.112696734
3	-7.994922879	-0.001707515
4	-7.995026371	-0.000103492
5	-7.995026742	-0.000000372

[a] Using TRS4 for density matrix purification, EPS_SCF=1.0E-6, EPS_FILTER=1.0E-7.

[b] Extrapolation of Kohn-Sham matrix, K, by mixing two previous K matrices using a MIXING_FRACTION A=0.45 ($K'=A*K+(1-A)*K_{old}$).

[c] Extrapolation of Kohn-Sham matrix using DIIS. The DIIS buffer, N_{buffer}=4, INI_DIIS=2 (INI_DIIS, LS_SCF iteration cycle to start DIIS), EPS_DIIS=1.0 (threshold on convergence to start DIIS).

Table 2: Comparison of single point energy convergence calculations on open shell CHO using linear scaling DFT[a] with (XC,BS)=(PADE,DZVP-GTH-PADE) with and without DIIS.

Without DIIS[b]		
SCF Nr	Energy (au)	ΔE (au)
1	-22.108361245	-22.108361245
2	-22.124336954	-0.015975710
3	-22.127188054	-0.002851100
4	-22.129079966	-0.001891912
5	-22.130664685	-0.001584718
6	-22.131860922	-0.001196237
	. . .	. . .
26	-22.136949498	-0.000018187
27	-22.136964372	-0.000014874
28	-22.136976384	-0.000012012
29	-22.136986196	-0.000009812
With DIIS[c]		
SCF Nr	Energy (au)	ΔE (au)
1	-22.108361245	-22.108361245
2	-22.124336954	-0.015975710
3	-22.131885667	-0.007548713
4	-22.136969150	-0.005083482
5	-22.137025506	-0.000056356
6	-22.137027991	-0.000002485

[a] Using TRS4 for density matrix purification, EPS_SCF=1.0E-6, EPS_FILTER=1.0E-7.

[b] Extrapolation of Kohn-Sham matrix, K, by mixing two previous K matrices using a MIXING_FRACTION A=0.45 ($K'=A*K+(1-A)*K_{old}$).

[c] Extrapolation of Kohn-Sham matrix using DIIS. The DIIS buffer, N_{buffer}=4, INI_DIIS=2 (INI_DIIS, LS_SCF iteration cycle to start DIIS), EPS_DIIS=1.0 (threshold on convergence to start DIIS).

Next, we do a series of single point energy calculations using linear scaling DFTB[2,3,6,7] on polyethelene orthorhombic system with sizes 18432 atoms, 36846 atoms and 147456 atoms using N_{proc}=1024 cpus and 4096 cpus to check the whole performance of DIIS in linear scaling module. The results are shown in Figure (1) where we observe the following: using DIIS i) reduces the walltime and ii) does not affect linear scalability.

Figure (2) shows the effect of changing threshold parameters EPS_SCF and EPS_FILTER on the performance of a linear scaling DFTB calculation on amorphous polyethelene with 13500 atoms with and without using DIIS, respectively. The following is observed: i) using not so tight threshold on SCF (EPS_SCF=1E-07) the calculations with DIIS are more accurate ($E-E_{exact} \approx 0$) than without using DIIS. However, using DIIS the walltime is a bit larger than without using DIIS; ii) using tight threshold on SCF (EPS_SCF=1E-10) both approaches with and without DIIS are very accurate. However, using DIIS shows less walltime than without using DIIS.

The results of another test to check for accuracy when using DIIS in linear scaling is shown in Table (3) where we compare walltime and precision of LS-DFTB calculations on polyethelene (PE) with 4000 atoms and 13500 atoms against the results from a very tight setting using diagonalization method. For the case of PE amorphous with 4000 atoms we see that: i) using default settings in LS calculation (EPS_SCF=1E-7,EPS_FILTER=1E-6,MIXING_FRACTION=0.45) with TRS4 as density matrix purification method the walltime is very short compared to the timing from using diagonalization method (e.g. 19 sec. as opposed to 127 sec.) but it is not very precise (e.g. error=0.001 a.u. compared to diagonalization energy); ii) changing to a MIXING_FRACTION=0.75 and keeping the other default settings one increases precision to energy convergence but the walltime is large compared to diagonalization method (e.g. 203 sec as opposed to 127 sec.); iii) using DIIS it converges to the same energy obtained with diagonalization method at a slightly larger walltime (e.g 142 sec as opposed to 127 sec). For the case of PE amorphous with 13500 atoms we see that: i) using default settings in LS calculation (EPS_SCF=1E-7,EPS_FILTER=1E-6,MIXING_FRACTION=0.45) with TRS4 as density matrix purification method the walltime is very short compared to the timing from using diagonalization method (e.g. 43 sec. as opposed to 1662 sec.) but it is not very precise (e.g. error=0.004 a.u. compared to diagonalization energy); ii) changing to a MIXING_FRACTION=0.75 and keeping the other default settings one increases precision to energy convergence and reduces significatively the walltime compared to diagonalization method (e.g. 438 sec as opposed to 1662 sec.); iii) using DIIS it converges to the same energy obtained with diagonalization method at a short walltime (e.g 338 sec as opposed to 1662 sec). It is important to point out that even there is an alternative way to reduce walltime and increase precision by choosing the appropriate MIXING_FRACTION value for a given molecular system this approach is not recommendable since finding such value would require sending several calculations to find it by trial.

Table 3: Performance single energy point calculations using LS-DFTB with different settings: i) TRS4 (defaults); ii) TRS4 (modified); iii) TRS4+DIIS. A comparison is made on precision and walltime against Diagonalization method (Diag) which is calculated using tight settings.

	PE Amorphous 4000 atoms (512 proc)		PE Amorphous 13500 atoms (1024 proc)	
Method	Energy (a.u.)	Time (sec)	Energy (a.u.)	Time (sec)
Diag[a]	-3222.031989	127	-10874.357270	1662
TRS4(defaults)[b]	-3222.030909	19	-10874.353634	43
TRS4(modified)[c]	-3222.031989	203	-10874.357270	438
TRS4+DIIS[d]	**-3222.031989**	**142**	**-10874.357269**	**335**

[a] Diagonalization method (Diag) uses tight settings.
[b] TRS4 (defaults) uses the following settings: EPS_SCF=1E-7,EPS_FILTER=1E-6, MIXING_FRACTION=0.45
[c] TRS4 (modified) uses all default settings except MIXING_FRACTION=0.75.
[d] TRS4+DIIS uses the following settings: EPS_SCF=1E-9, EPS_FILTER=1E-8, N_{buffer}=4, INI_DIIS=2 and EPS_DIIS=1.0.

Finally, we do series of tests on carbon nanotube with 3200 atoms whose very small band gap of 0.00409 a.u. challenges energy convergence methods. Table (4) shows the results on doing single point energy calculation using LS-DFTB and we compared performance against a diagonalization calculation done using tight settings. We obtain the

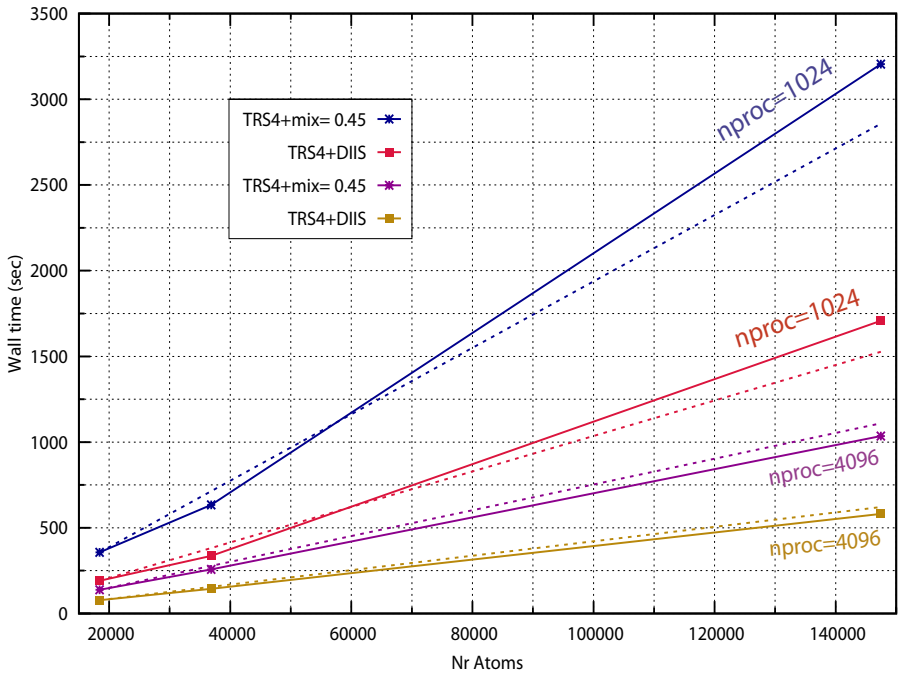

Figure 1: Showing walltimes from LS-DFTB single point energy calculations: i) without using DIIS (in blue and purple curves labeled TRS4+mix) and ii) using DIIS (in red and yellow curves labeled TRS4+DIIS). We observe that adding DIIS reduces walltime without affecting original linear scalability as shown in this graph obtained from a series of calculations on polyethelene orthorhombic system of sizes 18432 atoms, 36864 atoms and 147456 atoms using N_{proc}=1024 and 4096, respectively. The dotted lines mean ideal linear scalability while the continued line are interpolations of three sets of calculations. The calculations without DIIS use TRS4 for density matrix purification, EPS_SCF=1.0E-10, EPS_FILTER=1.0E-7 and MIXING_FRACTION A=0.45. The calculations with DIIS use TRS4 for density matrix purification, EPS_SCF=1.0E-10, EPS_FILTER=1.0E-7 and MIXING_FRACTION A=0.45 in the first SCF cycle. The DIIS settings were: N_{buffer}=4, INI_DIIS=2 (INI_DIIS, LS_SCF iteration cycle to start DIIS), EPS_DIIS=1.0 (threshold on convergence to start DIIS).

following: i) using TRS4 with defaults settings of EPS_SCF=1E-7, EPS_FILTER=1E-6 and MIXING_FRACTION=0.45 (used only in first SCF cycle) with DIIS (N_{buffer}=4, INI_DIIS=2 and EPS_DIIS=1.0) there is no energy convergence; ii) using TRS4 with defaults settings of EPS_SCF=1E-7, EPS_FILTER=1E-6 but changing the MIXING_FRACTION =0.05 one reaches energy convergence with E=-5545.777781 which differs from diagonalization value by 0.000023 a.u. and the wall time is much larger (766 sec. as opposed to 491 sec.); iii) using TRS4 with tight settings EPS_SCF=1E-9, EPS_FILTER=1E-8 but modified MIXING_FRACTION=0.05 no energy convergence is achieved; iv) using

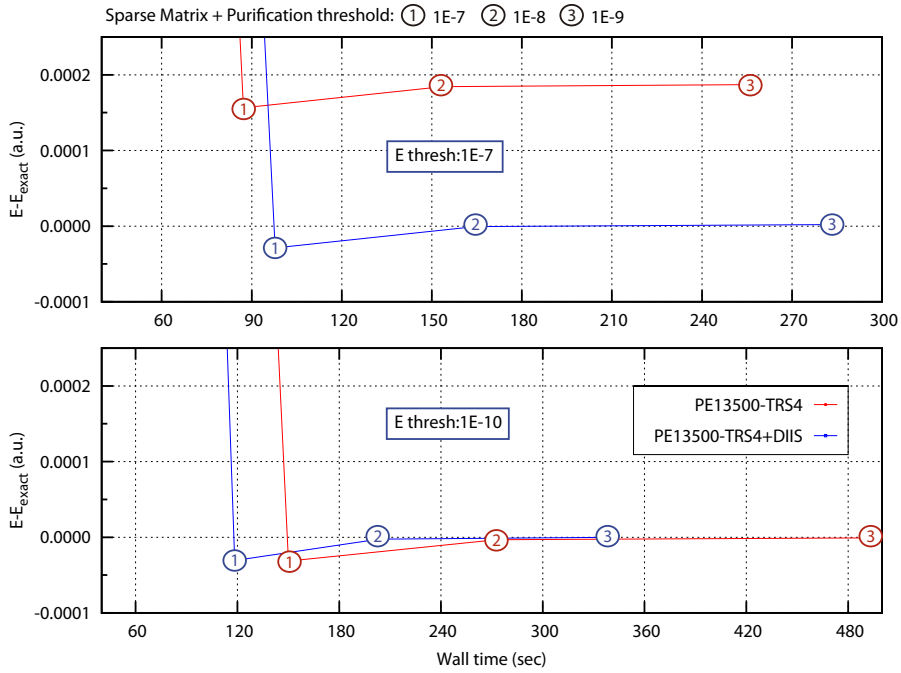

Figure 2: Performance of DIIS within linear scaling DFTB. Showing single point energy calculations on polyethylene amorphous system consisting of 13500 atoms for threshold settings:i) EPS_SCF=1E-7, 1E-10 (EPS_SCF is threshold in SCF energy convergence) and ii)EPS_FILTER= 1E-7, 1E-8, 1E-9 (EPS_FILTER is threshold in used density matrix purification method TRS4 and it is also threshold for controlling number sparse matrix multiplications). E_{exact} in the vertical label is the converged energy at very tight setting (e.g. EPS_SCF=1E-10 EPS_FILTER=1E-09) using DIIS which is known to be identical to a converged energy using diagonalization method.

TRS4 with settings indicated in ii) but in addition using DIIS one converges to about the same energy obtained in diagonalization method but a larger walltime (1266 sec. as opposed to 491 sec.).

Table 4: Performance single energy point calculations using LS-DFTB with different settings on Carbon nanotube (CNT) of 3200 atoms size with 512 processors: i) TRS4 (defaults)+DIIS; ii) TRS4 (modified); iii) TRS4 (modified) tight; iv) TRS4 (modified) + DIIS. A comparison is made on precision and wall-time against Diagonalization method (Diag) which is calculated using tight settings.

Method	Energy (a.u.)	Time (sec)
Diag[a]	-5545.777804	491
TRS4(defaults)+DIIS[b]	no convergence	
TRS4(modified)[c]	-5545.777781	766
TRS4(modified) tight [d]	no convergence	
TRS4(modified)+DIIS [e]	**-5545.777803**	**1266**

[a] Diagonalization method (Diag) uses tight settings.

[b] TRS4 (defaults)+DIIS uses EPS_SCF=1E-7, EPS_FILTER=1E-6 and MIXING_FRACTION=0.45 (used only in first SCF cycle) with DIIS (N_{buffer}=4, INI_DIIS=2 and EPS_DIIS=1.0)

[c] TRS4 (modified) uses EPS_SCF=1E-7, EPS_FILTER=1E-6 but changing the MIXING_FRACTION =0.05.

[d] TRS4 (modified) tight uses uses EPS_SCF=1E-9, EPS_FILTER=1E-8 but changing the MIXING_FRACTION =0.05.

[d] TRS4 (modified)+DIIS uses EPS_SCF=1E-7, EPS_FILTER=1E-6 but changing the MIXING_FRACTION =0.05 and with DIIS settings N_{buffer}=4, INI_DIIS=2 and EPS_DIIS=1.0.

4 Summary and conclusions

We have given details on implementation of **D**irect **I**nversion of the **I**terative **S**ubspace (**DIIS**) in linear scaling module in software CP2K. From the tests shown we see that DIIS accelerate energy convergence by reducing the number of SCF cycles ensuring higher precision and without affecting linear scalability. One could gain advantage of using DIIS on applying it to do electronic structure calculations of molecular systems that entail plenty of SCF cycles to converge gaining reduction of the number of SCF cycles.

Acknowledgments

This work has been possible with the support of U.S. Army Research Laboratory DoD Supercomputing Resource Center who provided me access to CRAY computers Copper and Garnet and gave me technical support on several computational issues. I would like to thank to Dr. Mathias Krack and Dr. Joost VandeVondele, CP2K developers, who guided me on understanding technical details on the implementation of DIIS and

usage of sparse matrix DBCSR library. Also, I want to thank Dr. Gary Kedziora and Dr. Jonathan Mullin for their helpful orientation and support on computational issues regarding compilation of CP2K and calculations on molecular systems. Finally, I want to give many thanks to Dr. Jan W. Andzelm from ARL-APG and Prof. Mark Robbins from Physics and Astronomy Department at Johns Hopkins University for their guidance on polyethelene research area.

References

[1] Borštnik, U. and VandeVondele, J. and Weber, V. and Hutter, J. "Sparse matrix multiplication: The distributed block-compressed sparse row library" Parallel Computing, **40**, 47–58, (2014)

[2] Elsner, J. and Haugk, M. and Frauenheim, T. and Suhai, S. and Seifert, G. "Self-consistent-charge density-functional tight-binding method for simulations of complex materials properties" Phys. Rev. B, **58**, 7260, (1998)

[3] Elstner, M. and Hobza, P. and Frauenheim, T. and Suhai, S. and Kaxiras, E., "Hydrogen bonding and stacking interactions of nucleic acid base pairs: a density-functional-theory based treatment" J. Chem. Phys., **114**, 5149–5155, (2001)

[4] Goedecker, S and Teter, M and Hutter, J., "Separable dual-space Gaussian pseudopotentials", Phys. Rev. B, **54**, 1703-1710, (1996)

[5] Harrison, R. J., "Krylov Subspace Accelerated Inexact Newton Method for Linear and Nonlinear Equations", J. Comput. Chem., **25**, 328–334, (2004)

[6] Haugk, M. and Elsner, J. and Heine, Th. and Frauenheim, Th. and Seifert, G., "A parallel code for a self-consistent charge density functional based tight binding method: Total energy calculations for extended systems", Comput. Mat. Sci., **13**, 239–251, (1999)

[7] Hutter, J. and Iannuzzi, M. and Schiffmann, F. and VandeVondele, J., "CP2K: atomistic simulations of condensed matter systems", Comput. Mol. Sci., **4**, 15–25, (2014)

[8] Kohn, W., Phys. Rev. Lett., **76**, 3168, (1996)

[9] Niklasson, A. and Tymczak, C. and Challacombe, M., "Trace resetting density matrix purification in O (N) self-consistent-field theory", J. Chem. Phys., **118**, 8611–8620, (2003)

[10] Prodan, E. and Kohn, W., Proc. Natl. Acad. Sci. U.S.A., **102**, 11635, (2005)

[11] Pulay, P., "Convergence acceleration of iterative sequences. The case of SCF iteration", Chem. Phys. Lett., **73**, 393–398, (1980).

[12] Pulay, P., “Improved SCF Convergence Acceleration”, J. Comput. Chem., **3**, 556–560, (1982)

[13] VandeVondele, J. and Krack, M. and Mohamed, F. and Parrinello, M. and Chassaing, T. and Hutter, J., “Quickstep: Fast and accurate density functional calculations using a mixed Gaussian and plane waves approach”, Comput. Phys.Comm., **167**, 103–128, (2005)

[14] VandeVondele, J. and Borštnik, U. and Hutter, J., “Linear scaling self-consistent field calculations with millions of atoms in the condensed phase”, J. Chem. Theory Comput., **8**, 3565–3573, (2012)

A Modified ls_scf_main routine

```
SUBROUTINE ls_scf_main(qs_env,ls_scf_env,error)
  TYPE(qs_environment_type), POINTER       :: qs_env
  TYPE(ls_scf_env_type)                    :: ls_scf_env
  TYPE(cp_error_type), INTENT(INOUT)       :: error

  CHARACTER(len=*), PARAMETER :: routineN = 'ls_scf_main', &
    routineP = moduleN//':'//routineN

  INTEGER                                  :: handle, iscf, ispin, &
                                              nelectron_spin_real, nmixing, &
                                              nspin, unit_nr
  LOGICAL                                  :: check_convergence, diis_step, &
                                              should_stop
  REAL(KIND=dp)                            :: energy_diff, energy_new, &
                                              energy_old, eps_diis, t1, t2
  TYPE(cp_dbcsr_p_type), DIMENSION(:), &
    POINTER                                :: matrix_ks, matrix_s
  TYPE(cp_dbcsr_type), ALLOCATABLE, &
    DIMENSION(:)                           :: matrix_ks_deviation, &
                                              matrix_mixing_old
  TYPE(cp_logger_type), POINTER            :: logger
  TYPE(qs_diis_buffer_type_sparse), &
    POINTER                                :: diis_buffer

  CALL timeset(routineN,handle)

  ! get a useful output_unit
  logger => cp_error_get_logger(error)
  IF (logger%para_env%mepos==logger%para_env%source) THEN
     unit_nr=cp_logger_get_default_unit_nr(logger,local=.TRUE.)
  ELSE
     unit_nr=-1
  ENDIF

  nspin=ls_scf_env%nspins
```

```
! old quantities, useful for mixing
ALLOCATE(matrix_mixing_old(nspin), matrix_ks_deviation(nspin))
DO ispin=1,nspin
   CALL cp_dbcsr_init(matrix_mixing_old(ispin),error=error)
   CALL cp_dbcsr_create(matrix_mixing_old(ispin),&
                        template=ls_scf_env%matrix_ks(ispin), &
                        error=error)

   CALL cp_dbcsr_init(matrix_ks_deviation(ispin),error=error)
   CALL cp_dbcsr_create(matrix_ks_deviation(ispin), &
                        template=ls_scf_env%matrix_ks(ispin), &
                        error=error)
   CALL cp_dbcsr_set(matrix_ks_deviation(ispin),0.0_dp,error=error)
ENDDO
ls_scf_env%homo_spin(:) = 0.0_dp
ls_scf_env%lumo_spin(:) = 0.0_dp

energy_old=0.0_dp
IF(ls_scf_env%scf_history%istore>0)energy_old=ls_scf_env%energy_init
check_convergence=.TRUE.
iscf=0
IF (ls_scf_env%ls_diis) THEN
 diis_step = .FALSE.
 eps_diis  = ls_scf_env%eps_diis
 nmixing   = ls_scf_env%nmixing
 NULLIFY(diis_buffer)
 IF (.NOT.ASSOCIATED(diis_buffer)) THEN
    CALL qs_diis_b_create_sparse(diis_buffer, &
                                 nbuffer=ls_scf_env%max_diis,  &
                                 error=error)
 END IF
 CALL qs_diis_b_clear_sparse(diis_buffer,error=error)
 CALL get_qs_env(qs_env,matrix_s=matrix_s,error=error)
END IF
! the real SCF loop
DO

  ! check on max SCF or timing/exit
  CALL external_control(should_stop,"SCF", &
                        start_time=qs_env%start_time, &
                        target_time=qs_env%target_time, &
                        error=error)
  IF (should_stop .OR. iscf>=ls_scf_env%max_scf) THEN
     IF (unit_nr>0) WRITE(unit_nr,'(T2,A)') "SCF not converged! "
     EXIT
  ENDIF

  t1 = m_walltime()
  iscf=iscf+1
```

```
! first get a copy of the current KS matrix
CALL get_qs_env(qs_env, matrix_ks=matrix_ks, error=error)
DO ispin=1,nspin
   CALL matrix_qs_to_ls(ls_scf_env%matrix_ks(ispin),&
                        matrix_ks(ispin)%matrix,    &
                        ls_scf_env%ls_mstruct,error=error)
   IF (ls_scf_env%has_s_preconditioner) THEN
       CALL apply_matrix_preconditioner(&
                      ls_scf_env%matrix_ks(ispin),"forward", &
                      ls_scf_env%matrix_bs_sqrt, &
                      ls_scf_env%matrix_bs_sqrt_inv,error)
   ENDIF
   CALL cp_dbcsr_filter(ls_scf_env%matrix_ks(ispin), &
                        ls_scf_env%eps_filter,error=error)
ENDDO
! run curvy steps if required.
! Needs an idempotent DM (either perification or restart)
IF((iscf>1.OR.ls_scf_env%scf_history%istore>0).AND. &
    ls_scf_env%curvy_steps) THEN
   CALL dm_ls_curvy_optimization(ls_scf_env,energy_old, &
                                 check_convergence,error)
ELSE
   ! turn the KS matrix in a density matrix
   DO ispin=1,nspin
      IF (iscf==1) THEN
         ! initialize the mixing matrix with the current state if needed
         CALL cp_dbcsr_copy(matrix_mixing_old(ispin),    &
                            ls_scf_env%matrix_ks(ispin), &
                            error=error)
      ELSE
       IF (ls_scf_env%ls_diis) THEN ! ------- IF-DIIS+MIX--- START
        IF (diis_step.and.(iscf-1).ge.ls_scf_env%iter_ini_diis) THEN
         IF (unit_nr>0) THEN
          WRITE(unit_nr,'(A)') &
            '*********************************************************'
          WRITE(unit_nr,'(A,2(I3,A),L,A)') &
            ' Using DIIS mixed KS:  (iscf,INI_DIIS,DIIS_STEP)=(' , &
            iscf,',',ls_scf_env%iter_ini_diis,',',diis_step,')'
          WRITE(unit_nr,'(A)') &
            ' KS_nw= DIIS-Linear-Combination-Previous KS matrices'
          WRITE(unit_nr,'(A)') &
            '*********************************************************'
         ENDIF
         CALL cp_dbcsr_copy(matrix_mixing_old(ispin),   & ! out
                            ls_scf_env%matrix_ks(ispin),& ! in
                            error=error)
        ELSE
         IF (unit_nr>0) THEN
          WRITE(unit_nr,'(A)') &
```

```
      '*******************************************************'
    WRITE(unit_nr,'(A,F5.3,A,I3)') &
      ' Using MIXING_FRACTION=',ls_scf_env%mixing_fraction, &
      ' to mix KS matrix:  iscf=',iscf
    WRITE(unit_nr,'(2(A,F5.3),A)') &
      ' KS_nw=',ls_scf_env%mixing_fraction,'*KS + ', &
       1.0_dp-ls_scf_env%mixing_fraction,'*KS_old'
    WRITE(unit_nr,'(A)') &
      '*******************************************************'
   ENDIF
   ! perform the mixing of ks matrices
   CALL cp_dbcsr_add(matrix_mixing_old(ispin)   ,       &
                     ls_scf_env%matrix_ks(ispin),       &
                     1.0_dp-ls_scf_env%mixing_fraction, &
                     ls_scf_env%mixing_fraction,        &
                     error=error)
  ENDIF
 ELSE ! otherwise
   IF (unit_nr>0) THEN
    WRITE(unit_nr,'(A)') &
     '*******************************************************'
    WRITE(unit_nr,'(A,F5.3,A,I3)') &
     ' Using MIXING_FRACTION=', ls_scf_env%mixing_fraction, &
     ' to mix KS matrix:  iscf=',iscf
    WRITE(unit_nr,'(2(A,F5.3),A)') &
     ' KS_nw=',ls_scf_env%mixing_fraction,'*KS + ', &
     1.0_dp-ls_scf_env%mixing_fraction,'*KS_old'
    WRITE(unit_nr,'(A)') &
    '*******************************************************'
   ENDIF
   ! perform the mixing of ks matrices
   CALL cp_dbcsr_add(matrix_mixing_old(ispin)   ,       &
                     ls_scf_env%matrix_ks(ispin),       &
                     1.0_dp-ls_scf_env%mixing_fraction, &
                     ls_scf_env%mixing_fraction,        &
                     error=error)
 ENDIF ! ------- IF-DIIS+MIX--- END
ENDIF

! compute the density matrix that matches it
! we need the proper number of states
nelectron_spin_real=ls_scf_env%nelectron_spin(ispin)
IF (ls_scf_env%nspins==1) nelectron_spin_real=nelectron_spin_real/2

SELECT CASE(ls_scf_env%purification_method)
CASE(ls_scf_ns)
  CALL density_matrix_sign(         &
        ls_scf_env%matrix_p(ispin), &
        ls_scf_env%mu_spin(ispin),  &
        ls_scf_env%fixed_mu,        &
```

```
            matrix_mixing_old(ispin),   &
            ls_scf_env%matrix_s,        &
            ls_scf_env%matrix_s_inv,    &
            nelectron_spin_real,        &
            ls_scf_env%eps_filter,error)
    CASE(ls_scf_tc2)
      CALL density_matrix_tc2(           &
            ls_scf_env%matrix_p(ispin),  &
            matrix_mixing_old(ispin),    &
            ls_scf_env%matrix_s_sqrt_inv,&
            nelectron_spin_real,         &
            ls_scf_env%eps_filter,       &
            ls_scf_env%homo_spin(ispin), &
            ls_scf_env%lumo_spin(ispin), &
            non_monotonic=ls_scf_env%non_monotonic, &
            eps_lanczos=ls_scf_env%eps_lanczos, &
            max_iter_lanczos=ls_scf_env%max_iter_lanczos,&
            error=error)
    CASE(ls_scf_trs4)
      CALL density_matrix_trs4(          &
            ls_scf_env%matrix_p(ispin),  &
            matrix_mixing_old(ispin),    &
            ls_scf_env%matrix_s_sqrt_inv,&
            nelectron_spin_real,         &
            ls_scf_env%eps_filter,       &
            ls_scf_env%homo_spin(ispin), &
            ls_scf_env%lumo_spin(ispin), &
            ls_scf_env%mu_spin(ispin),   &
            dynamic_threshold=ls_scf_env%dynamic_threshold,&
            matrix_ks_deviation=matrix_ks_deviation(ispin), &
            eps_lanczos=ls_scf_env%eps_lanczos, &
            max_iter_lanczos=ls_scf_env%max_iter_lanczos,&
            error=error)
    END SELECT

    IF (ls_scf_env%has_s_preconditioner) THEN
        CALL apply_matrix_preconditioner(            &
                ls_scf_env%matrix_p(ispin),"forward", &
                ls_scf_env%matrix_bs_sqrt,            &
                ls_scf_env%matrix_bs_sqrt_inv,        &
                error)
    ENDIF
    CALL cp_dbcsr_filter(ls_scf_env%matrix_p(ispin),  &
                         ls_scf_env%eps_filter,error=error)

    IF (ls_scf_env%nspins==1) &
       CALL cp_dbcsr_scale(ls_scf_env%matrix_p(ispin),&
                           2.0_dp,error=error)

ENDDO
```

```
      END IF

      ! compute the corresponding new energy KS matrix and new energy
      CALL ls_scf_dm_to_ks(qs_env,ls_scf_env,energy_new,error)

      ! report current SCF loop
      energy_diff=energy_new-energy_old
      energy_old=energy_new

      t2 = m_walltime()
      IF (unit_nr>0) THEN
         WRITE(unit_nr,*)
         WRITE(unit_nr,'(T2,A,I6,F20.9,F20.9,F12.6)') &
               "SCF",iscf, energy_new,energy_diff, t2-t1
         WRITE(unit_nr,*)
      ENDIF

      ! exit criterium on the energy only for the time being
      IF (check_convergence.AND. &
          ABS(energy_diff)<ls_scf_env%eps_scf*ls_scf_env%nelectron_total) EXIT

      IF (ls_scf_env%ls_diis) THEN
! diis_buffer, buffer with
!    1) Kohn-Sham history matrix,
!    2) KS error history matrix (f=KPS-SPK),
!    3) B matrix (for finding DIIS weighting coefficients)
       CALL qs_diis_b_step_4lscf(  &
           diis_buffer,            &
           qs_env,ls_scf_env,      &
           unit_nr,                &
           iscf,diis_step,eps_diis,&
           nmixing,                &
           matrix_s(1)%matrix,     &
           ls_scf_env%eps_filter,  &
           error)
      ENDIF

    ENDDO

    ! free storage
    IF (ls_scf_env%ls_diis) THEN
       CALL qs_diis_b_release_sparse(diis_buffer,error)
    ENDIF
    DO ispin=1,nspin
       CALL cp_dbcsr_release(matrix_mixing_old(ispin),error=error)
       CALL cp_dbcsr_release(matrix_ks_deviation(ispin),error=error)
    ENDDO
    DEALLOCATE(matrix_mixing_old, matrix_ks_deviation)

    CALL timestop(handle)
```

```
  END SUBROUTINE ls_scf_main
```

B qs_diis_b_step_4lscf routine

```
  SUBROUTINE qs_diis_b_step_4lscf(    &
                  diis_buffer,        &
                  qs_env,ls_scf_env,  &
                  unit_nr,iscf,       &
                  diis_step,eps_diis,&
                  nmixing,s_matrix,   &
                  threshold,          &
                  error)
! Note.- Input: ls_scf_env%matrix_p(ispin) , Density Matrix
!               matrix_ks (from qs_env)    , Kohn-Sham Matrix  (IN/OUT)

    TYPE(qs_diis_buffer_type_sparse), &
      POINTER                                 :: diis_buffer
    TYPE(qs_environment_type), POINTER        :: qs_env
    TYPE(ls_scf_env_type)                     :: ls_scf_env
    INTEGER, INTENT(IN)                       :: unit_nr, iscf
    LOGICAL, INTENT(OUT)                      :: diis_step
    REAL(KIND=dp), INTENT(IN)                 :: eps_diis
    INTEGER, INTENT(IN), OPTIONAL             :: nmixing
    TYPE(cp_dbcsr_type), OPTIONAL             :: s_matrix
    REAL(KIND=dp), INTENT(IN)                 :: threshold
    TYPE(cp_error_type), INTENT(INOUT)        :: error

    CHARACTER(LEN=*), PARAMETER :: routineN = 'qs_diis_b_step_4lscf', &
      routineP = moduleN//':'//routineN
    REAL(KIND=dp), PARAMETER :: eigenvalue_threshold = 1.0E-12_dp

    INTEGER                                   :: handle, ib, ispin, istat, jb, &
                                                 my_nmixing, nb, nb1, nspin
    REAL(KIND=dp)                             :: error_max, tmp
    REAL(KIND=dp), ALLOCATABLE, DIMENSION(:) :: ev
    REAL(KIND=dp), DIMENSION(:, :), POINTER  :: a, b
    TYPE(cp_dbcsr_p_type), DIMENSION(:), &
      POINTER                                 :: matrix_ks
    TYPE(cp_dbcsr_type)                       :: matrix_KSerr_t, matrix_tmp
    TYPE(cp_dbcsr_type), POINTER              :: new_errors, old_errors, &
                                                 parameters
    TYPE(cp_logger_type), POINTER             :: logger
    TYPE(cp_para_env_type), POINTER           :: para_env

    CALL timeset(routineN,handle)
    nspin=ls_scf_env%nspins
    diis_step = .FALSE.
    my_nmixing = 2
```

```
    IF (PRESENT(nmixing)) my_nmixing = nmixing
    NULLIFY (new_errors,old_errors,parameters,a,b)
    logger => cp_error_get_logger(error)
    ! Quick return, if no DIIS is requested
    IF (diis_buffer%nbuffer < 1) THEN
      CALL timestop(handle)
      RETURN
    END IF

! Getting current Kohn-Sham matrix from qs_env
    CALL get_qs_env(qs_env,              &
                    para_env=para_env,   &
                    matrix_ks=matrix_ks, &
                    error=error)
    CALL qs_diis_b_check_i_alloc_sparse( &
                          diis_buffer,  &
                          ls_scf_env,   &
                          nspin,        &
                          error)
    error_max = 0.0_dp

    ib = MODULO(diis_buffer%ncall,diis_buffer%nbuffer) + 1
    diis_buffer%ncall = diis_buffer%ncall + 1
    nb = MIN(diis_buffer%ncall,diis_buffer%nbuffer)
! Create scratch arrays
    CALL cp_dbcsr_init(matrix_tmp,error=error)
    CALL cp_dbcsr_create(matrix_tmp, &
                         template=ls_scf_env%matrix_ks(1),&
                         matrix_type='N',error=error)
    CALL cp_dbcsr_set(matrix_tmp,0.0_dp,error=error)     ! reset matrix
    CALL cp_dbcsr_init(matrix_KSerr_t,error=error)
    CALL cp_dbcsr_create(matrix_KSerr_t,&
                         template=ls_scf_env%matrix_ks(1),&
                         matrix_type='N',error=error)
    CALL cp_dbcsr_set(matrix_KSerr_t,0.0_dp,error=error) ! reset matrix

    DO ispin=1,nspin ! ------ Loop-ispin----START

      new_errors => diis_buffer%error(ib,ispin)%matrix
      parameters => diis_buffer%parameter(ib,ispin)%matrix
      ! Copy the Kohn-Sham matrix K to the DIIS buffer
      CALL cp_dbcsr_copy(parameters,              & ! out
                         matrix_ks(ispin)%matrix, & ! in
                         error=error)

        IF (PRESENT(s_matrix)) THEN ! if-s_matrix ---------- START
! Calculate Kohn-Sham error (non-orthogonal)= K*P*S-(K*P*S)^T
! matrix_tmp = P*S
            CALL cp_dbcsr_multiply("N", "N",         &
                   1.0_dp,ls_scf_env%matrix_p(ispin),&
```

```
                        s_matrix,                  &
                  0.0_dp,matrix_tmp,               &
                  filter_eps=threshold, error=error)
! new_errors= K*P*S
            CALL cp_dbcsr_multiply("N", "N",         &
                  1.0_dp,matrix_ks(ispin)%matrix,    &
                        matrix_tmp,                  &
                  0.0_dp,new_errors,                 &
                  filter_eps=threshold,error=error)
! matrix_KSerr_t= transpose(K*P*S)
            CALL cp_dbcsr_transposed(matrix_KSerr_t, &
                                     new_errors,     &
                                     error=error)
! new_errors=K*P*S-transpose(K*P*S)
            CALL cp_dbcsr_add(new_errors,       &
                              matrix_KSerr_t,   &
                              1.0_dp,-1.0_dp,   &
                              error=error)
        ELSE   ! if-s_matrix ---------- MID
! Calculate Kohn-Sham error (orthogonal)= K*P - P*K
! new_errors=K*P
            CALL cp_dbcsr_multiply("N", "N",         &
                  1.0_dp,matrix_ks(ispin)%matrix,    &
                        ls_scf_env%matrix_p(ispin),&
                  0.0_dp,new_errors,                 &
                  filter_eps=threshold,error=error)
! matrix_KSerr_t= transpose(K*P)
            CALL cp_dbcsr_transposed(matrix_KSerr_t, &
                                     new_errors,     &
                                     error=error)
! new_errors=K*P-transpose(K*P)
            CALL cp_dbcsr_add(new_errors,            &
                              matrix_KSerr_t,        &
                              1.0_dp,-1.0_dp,        &
                              error=error)
        END IF ! if-s_matrix ---------- END

       tmp=cp_dbcsr_maxabs(new_errors)
       error_max = MAX(error_max,tmp)

    END DO ! ------ Loop-ispin----END

    ! Check, if a DIIS step is appropiate

    diis_step = (diis_buffer%ncall >= my_nmixing)

    IF (unit_nr>0) THEN
      WRITE(unit_nr,'(A29,I3,A3,4(I3,A1))') &
          "DIIS: (ncall,nbuffer,ib,nb)=(",iscf,")=(", &
          diis_buffer%ncall,",",diis_buffer%nbuffer,",",ib,",",nb,")"
```

```
  WRITE(unit_nr,'(A57,I3,A3,L,A1,F10.8,A1,F4.2,A1,L,A1)')     &
      "DIIS: (diis_step,error_max,eps_diis,error_max<eps_diis)=(",&
      iscf,")=(",diis_step,",",error_max,",",eps_diis ,",", &
     (error_max<eps_diis),")"
  WRITE(unit_nr,'(A,A)') &
  "DIIS: diis_step=T : Perform DIIS  error_max<eps_diis=T :", &
  "Update DIIS buffer"
ENDIF

! Update the SCF DIIS buffer
IF (error_max < eps_diis) THEN
  b => diis_buffer%b_matrix
  DO jb=1,nb
    b(jb,ib) = 0.0_dp
    DO ispin=1,nspin
      old_errors => diis_buffer%error(jb,ispin)%matrix
      new_errors => diis_buffer%error(ib,ispin)%matrix
      CALL cp_dbcsr_trace(old_errors, &
                          new_errors, &
                          tmp,        & ! out : < f_i | f_j >
                          error=error)
      b(jb,ib) = b(jb,ib) + tmp
    END DO ! end-loop-ispin
    b(ib,jb) = b(jb,ib)
  END DO ! end-loop-jb
ELSE
  diis_step = .FALSE.
END IF

! Perform DIIS step
IF (diis_step) THEN
  nb1 = nb + 1
  ALLOCATE (a(nb1,nb1),STAT=istat)
  IF (istat /= 0) CALL stop_memory(routineN,moduleN,__LINE__,&
                                   "a",nb1*nb1*dp_size)
  ALLOCATE (b(nb1,nb1),STAT=istat)
  IF (istat /= 0) CALL stop_memory(routineN,moduleN,__LINE__,&
                                   "b",nb1*nb1*dp_size)
  ALLOCATE (ev(nb1),STAT=istat)
  IF (istat /= 0) CALL stop_memory(routineN,moduleN,__LINE__,&
                                   "ev",nb1*dp_size)
  ! Set up the linear DIIS equation system
  b(1:nb,1:nb) = diis_buffer%b_matrix(1:nb,1:nb)
  b(1:nb, nb1) = -1.0_dp
  b(nb1 ,1:nb) = -1.0_dp
  b(nb1 , nb1) =  0.0_dp
  ! Solve the linear DIIS equation system
  CALL diamat_all(b(1:nb1,1:nb1),ev(1:nb1),error=error)
  a(1:nb1,1:nb1) = b(1:nb1,1:nb1)
  DO jb=1,nb1
```

```
      IF (ABS(ev(jb)) < eigenvalue_threshold) THEN
        a(1:nb1,jb) = 0.0_dp
      ELSE
        a(1:nb1,jb) = a(1:nb1,jb)/ev(jb)
      END IF
    END DO ! end-loop-jb

    ev(1:nb) = MATMUL(a(1:nb,1:nb1),b(nb1,1:nb1))

    ! Update Kohn-Sham matrix
    IF (iscf.GE.ls_scf_env%iter_ini_diis) THEN  ! if-to-updateKS-- START

       IF (unit_nr>0) THEN
        WRITE(unit_nr,'(A40,I3)') &
        'DIIS: Updating Kohn-Sham matrix at iscf=',iscf
       ENDIF

      DO ispin=1,nspin
        CALL cp_dbcsr_set(matrix_ks(ispin)%matrix, & ! reset matrix
                          0.0_dp,                  &
                          error=error)
        DO jb=1,nb
           parameters => diis_buffer%parameter(jb,ispin)%matrix
           CALL cp_dbcsr_add(matrix_ks(ispin)%matrix,parameters, &
                             1.0_dp,-ev(jb),                     &
                             error=error)
        END DO ! end-loop-jb
      END DO ! end-loop-ispin
    ENDIF ! if-to-updateKS------ END

    DEALLOCATE (a,STAT=istat)
    IF (istat /= 0) CALL stop_memory(routineN,moduleN,__LINE__,"a")
    DEALLOCATE (b,STAT=istat)
    IF (istat /= 0) CALL stop_memory(routineN,moduleN,__LINE__,"b")
    DEALLOCATE (ev,STAT=istat)
    IF (istat /= 0) CALL stop_memory(routineN,moduleN,__LINE__,"ev")

  ELSE
    DO ispin=1,nspin
     parameters => diis_buffer%parameter(ib,ispin)%matrix
     CALL cp_dbcsr_copy(parameters,              & ! out
                        matrix_ks(ispin)%matrix, & ! in
                        error=error)
    ENDDO ! end-loop-ispin
  END IF
    CALL cp_dbcsr_release(matrix_tmp    ,error=error)
    CALL cp_dbcsr_release(matrix_KSerr_t,error=error)
    CALL timestop(handle)

END SUBROUTINE qs_diis_b_step_4lscf
```

C LS+DIIS input test for closed shell CH_4

```
&GLOBAL
  PROJECT ch4-DIIS_LSCF
  PRINT_LEVEL LOW
  RUN_TYPE ENERGY
&END GLOBAL
&FORCE_EVAL
  &DFT
    BASIS_SET_FILE_NAME ../../../data/BASIS_SET
    POTENTIAL_FILE_NAME ../../../data/GTH_POTENTIALS
    &MGRID
      CUTOFF 100
    &END MGRID
    &QS
      EPS_DEFAULT 1.0E-12
      LS_SCF
      &DISTRIBUTION
        BASIC_OPTIMIZATION .FALSE.
        BASIC_SPATIAL_OPTIMIZATION .TRUE.
      &END
    &END QS
    &XC
      &XC_FUNCTIONAL PADE
      &END XC_FUNCTIONAL
    &END XC
    &LS_SCF
      LS_DIIS       ! activate DIIS
      INI_DIIS 2    ! LS_SCF iteration cycle to start DIIS (default = 2)
      MAX_DIIS 4    ! DIIS buffer size (default = 4)
      EPS_DIIS 1.0  ! threshold on convergence to start DIIS (default = 0.1)
      MAX_SCF  40
      EPS_FILTER 1.0E-7
      EPS_SCF    1.0E-6
      MU        -0.20
      PURIFICATION_METHOD TRS4
      MIXING_FRACTION 0.45
      S_PRECONDITIONER ATOMIC
      REPORT_ALL_SPARSITIES OFF
      PERFORM_MU_SCAN OFF
    &END
   &END DFT
  &SUBSYS
    &CELL
      ABC 5.0 5.0 5.0
    &END CELL
```

```
    &COORD
     C  -0.000367    0.000000    0.000481
     H   0.638760   -0.000080    0.888272
     H   0.207743    0.893069   -0.597225
     H   0.207729   -0.892972   -0.597375
     H  -1.052031   -0.000017    0.303441
    &END COORD
    &KIND H
      BASIS_SET DZVP-GTH-PADE
      POTENTIAL GTH-PADE-q1
    &END KIND
    &KIND C
      BASIS_SET DZVP-GTH-PADE
      POTENTIAL GTH-PADE-q4
    &END KIND
  &END SUBSYS
&END FORCE_EVAL
```

D LS+DIIS input test for open shell CHO

```
&GLOBAL
  PROJECT cho-DIIS_LSCF
  PRINT_LEVEL LOW
  RUN_TYPE ENERGY
&END GLOBAL
&FORCE_EVAL
  &DFT
    CHARGE 0
    MULTIPLICITY 2
    UKS
    BASIS_SET_FILE_NAME ../../../data/BASIS_SET
    POTENTIAL_FILE_NAME ../../../data/GTH_POTENTIALS
    &MGRID
      CUTOFF 100
    &END MGRID
    &QS
      EPS_DEFAULT 1.0E-12
      LS_SCF
      &DISTRIBUTION
        BASIC_OPTIMIZATION .FALSE.
        BASIC_SPATIAL_OPTIMIZATION .TRUE.
      &END
    &END QS
    &XC
      &XC_FUNCTIONAL PADE
```

```
      &END XC_FUNCTIONAL
    &END XC
    &LS_SCF
      LS_DIIS       ! activate DIIS
      INI_DIIS 2    ! LS_SCF iteration cycle to start DIIS (default = 2)
      MAX_DIIS 4    ! DIIS buffer size (default = 4)
      EPS_DIIS 1.0  ! threshold on convergence to start DIIS (default = 0.1)
      MAX_SCF    40
      EPS_FILTER 1.0E-7
      EPS_SCF    1.0E-6
      MU        -0.20
      PURIFICATION_METHOD TRS4
      MIXING_FRACTION 0.45
      S_PRECONDITIONER ATOMIC
      REPORT_ALL_SPARSITIES OFF
      PERFORM_MU_SCAN OFF
    &END
   &END DFT
  &SUBSYS
    &CELL
      ABC 5.0 5.0 5.0
    &END CELL
    &COORD
     C     0.00000000   0.00000000  0.00000000
     O     1.18337200   0.00000000  0.00000000
     H    -0.63151821   0.94387462  0.00000000
    &END COORD
    &KIND H
      BASIS_SET DZVP-GTH-PADE
      POTENTIAL GTH-PADE-q1
    &END KIND
    &KIND C
      BASIS_SET DZVP-GTH-PADE
      POTENTIAL GTH-PADE-q4
    &END KIND
    &KIND O
      BASIS_SET DZVP-GTH-PADE
      POTENTIAL GTH-PADE-q6
    &END KIND
  &END SUBSYS
&END FORCE_EVAL
```